



Guía Definitiva: Commits Profesionales y Git Convencional

Esta guía reúne las mejores prácticas de la industria (basadas en el estándar *Conventional Commits*) para escribir historiales de Git limpios, legibles y profesionales.



1. La Regla de Oro: El límite de caracteres (50 / 72)

Un buen commit debe pensarse con la estructura de un correo electrónico: un **asunto** corto y directo, y un **cuerpo** explicativo si es necesario.



El Título (Asunto) -> Máximo 50 caracteres

- Es la primera línea de tu commit.
- **¿Por qué 50?** Porque plataformas como GitHub o la consola de Git cortan los títulos largos con puntos suspensivos (...).
- **Regla clave:** Debe empezar con minúscula después del prefijo y describir la acción en tiempo presente o imperativo (ej. `feat: add button` en lugar de `feat: added button`).



El Cuerpo (Descripción) -> Máximo 72 caracteres por línea

- Si el cambio es complejo, dejas una línea en blanco después del título y escribes un párrafo.
 - **¿Por qué 72?** Para que el texto no se deforme ni se desparrame cuando otro programador lo lea desde la terminal de comandos. Responde al *¿Por qué se hizo el cambio?* y no al *¿Qué se hizo?* (eso ya lo muestra el código).
-



2. Tipos de Commits (Prefijos Universales)

Usa siempre estos prefijos seguidos de dos puntos (:) para que cualquiera entienda tu trabajo de un vistazo:

Prefijo	¿Cuándo se usa?	Ejemplo real
feat	Una nueva característica o funcionalidad.	<code>feat: agregar botón para calcular división</code>
fix	Solución a un error (bug) en el código.	<code>fix: corregir error al dividir por cero</code>
docs	Solo cambios en la documentación (README, comentarios).	<code>docs: explicar uso de favicon en el HTML</code>
style	Cambios visuales, diseño, espacios o punto y coma.	<code>style: cambiar color de fondo a gris claro</code>

Prefijo	¿Cuándo se usa?	Ejemplo real
refactor	Cambios en el código que ni arreglan un bug ni añaden una función.	refactor: simplificar bucle de validación
test	Añadir pruebas que faltaban o corregir las existentes.	test: agregar prueba para el campo número 1
chore	Tareas de mantenimiento (actualizar librerías, configurar herramientas).	chore: actualizar versión de python en el proyecto

3. Más información y buenas prácticas

La anatomía de un commit perfecto

Si tuvieras que usar el título y el cuerpo juntos, se escribiría de la siguiente forma en la terminal:

Plaintext

```
feat: agregar selector dinámico para el botón de calcular
```

```
Se implementó el uso de IDs dinámicos para evitar colisiones  
en el DOM cuando se renderizan múltiples calculadoras en la  
misma vista de Automation.
```

Consejos extra para Testing y Automatización

- **Commits Atómicos:** Haz commits pequeños. Es preferible hacer 5 commits cortos que expliquen perfectamente el paso a paso (ej. uno para el HTML, uno para el CSS, uno para los selectores), que un solo commit gigante que diga `feat: hice todo el proyecto`.
- **Evita los mensajes basura:** Nunca uses mensajes como `fix: dsfjslkf`, `fix: ahora sí funciona` o `update`.